Verslag Scheduling Algoritmes

Amparo Walter en Malik Dutaev

Implementatie

We implementeren zes scheduling-algoritmes: FCFS, SJF, SRT, RR, HRRN en MLFQ. Elk algoritme erft van een gemeenschappelijke basisklasse (BaseScheduler), die de Scheduler-interface implementeert. Deze opzet ontkoppelt de processen van de scheduling-logica en vermijdt dubbele code. Afgewerkte processen worden centraal bijgehouden in een gedeelde wachtrij.

Elk proces wordt voorgesteld door een Process-klasse met de servicetijd, aankomsttijd, starttijd en eindtijd. Op basis daarvan berekenen we achteraf de genormaliseerde doorlooptijd (nTAT): (eindtijd − aankomsttijd) / servicetijd.

Niet-preëmptieve algoritmes

FCFS verwerkt processen in volgorde van aankomst via een LinkedList.

SJF gebruikt een PriorityQueue gesorteerd op servicetijd, zodat het proces met de kortste servicetijd altijd als eerste wordt uitgevoerd. Beide algoritmes delen dezelfde basislogica: als er geen actief proces is, wordt het volgende uit de wachtrij gehaald.

HRRN berekent bij elke proceswissel de responsverhouding voor elk wachtend proces: (wachttijd + servicetijd) / servicetijd. Het proces met de hoogste verhouding krijgt voorrang. Zo zorgt HRRN ervoor dat lange processen niet onbeperkt blijven wachten.

Preëmptieve algoritmes

SRT is een preëmptieve uitbreiding van SJF. Bij elke tijdseenheid vergelijkt het algoritme de resterende tijd van het actieve proces met die van het kortste wachtende proces. Als een nieuw aangekomen proces een kortere resterende tijd heeft, wordt de verstreken tijd opgeslagen in een ProcessEntry-record en wordt het actieve proces later hervat. RR werkt op een vergelijkbare manier, maar onderbreekt processen na een vaste tijdseenheid (tijdslice) in plaats van op basis van resterende tijd.

Multi-level feedback queue

We implementeren twee varianten van MLFQ.

MLFQWithAbsPriority geeft hogere wachtrijen absolute voorrang. Zodra een proces op niveau 0 beschikbaar is, wordt het lopende proces onmiddellijk onderbroken en teruggezet in zijn wachtrij. Processen zakken naar een lager niveau zodra ze hun

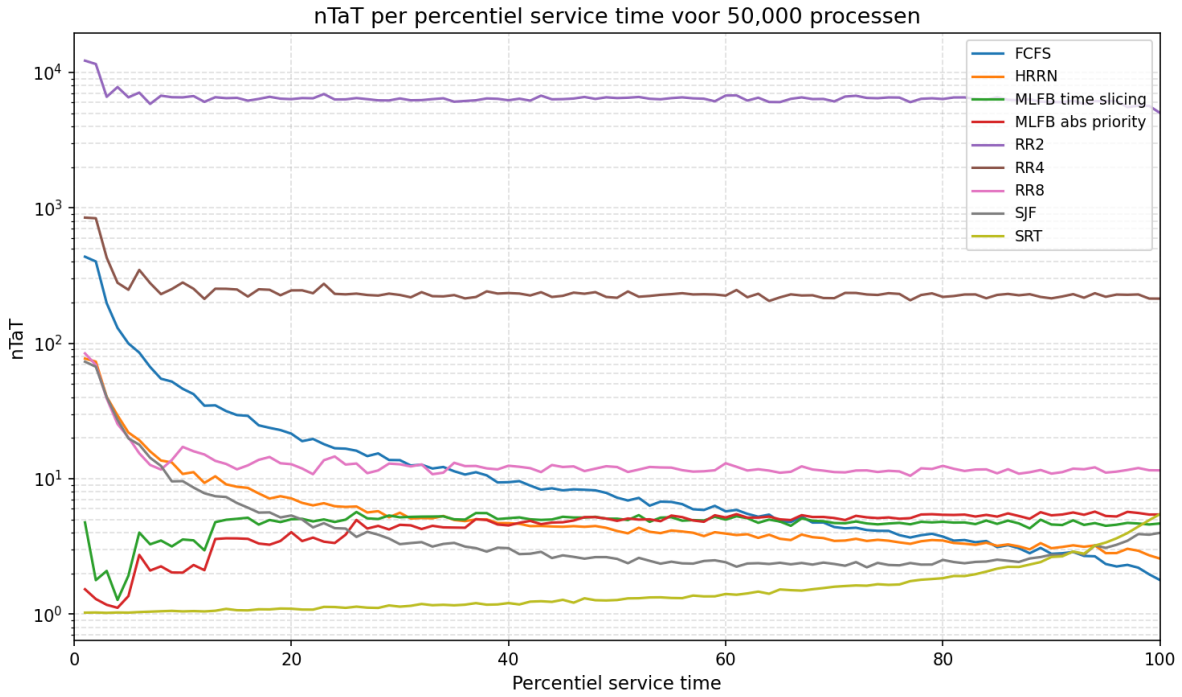
tijdslice hebben opgebruikt, totdat ze in de FCFS-wachtrij belanden. Dit is dan ook de laagste rang.

MLFQWithTimeSlicing voegt een budgetsysteem toe. Elke wachtrij krijgt een maximaal aantal cycli per ronde. Wanneer alle budgetten uitgeput zijn, begint een nieuwe ronde. Wachtrij 0 werkt als SRT in plaats van FCFS, wat de nTAT voor korte processen sterk verbetert.

Beide varianten bevatten aging: processen die langer dan een instelbare drempel wachten, worden één niveau omhooggeplaatst, zodat ze niet onbeperkt kunnen blijven wachten.

Evaluatie

We vergelijken de algoritmes op basis van de nTAT per percentiel van de servicetijd, gemeten over 50.000 processen. De y-as gebruikt een logaritmische schaal voor een duidelijker beeld van de spreiding.



SRT en SJF

SRT behaalt de laagste nTAT over bijna alle percentielen. De curve blijft vrijwel vlak met een lichte stijging aan het einde, wat aangeeft dat de doorlooptijd nauwelijks groter is dan de servicetijd. Dit komt doordat SRT continu het proces met de kortste resterende tijd uitvoert, waardoor wachttijden minimaal blijven.

SJF presteert goed voor korte processen, maar kan — anders dan SRT — lopende processen niet onderbreken. Een kort proces dat laat aankomt, moet daardoor

wachten tot het huidige proces volledig is afgewerkt. Dit verklaart de hogere nTAT bij lage percentielen in vergelijking met SRT.

FCFS en HRRN

FCFS levert een zeer hoge nTAT op voor de kortste processen. Als een kort proces toevallig na een lang proces aankomt, wacht het onevenredig lang — dit staat bekend als het konvooi-effect. Naarmate de servicetijd toeneemt, weegt de wachttijd relatief minder zwaar, waardoor de nTAT daalt.

HRRN vertoont een vergelijkbaar patroon, maar met een lagere startscore voor de kortste processen. De responsverhouding geeft namelijk voorrang aan korte processen die al lang wachten, waardoor het konvooi-effect wordt beperkt. Bij hogere percentielen convergeert HRRN naar dezelfde nTAT als SJF en MLFQ, omdat lange processen sowieso relatief weinig wachten.

MLFQ met absolute prioriteit en met tijdverdeling

MLFQ met absolute prioriteit scoort laag voor de kortste processen: een nieuw kort proces onderbreekt het lopende proces onmiddellijk, waardoor het heel snel wordt afgehandeld. Vanaf het vijfde percentiel stabiliseert de curve voor de rest van de verdeling.

MLFQ met tijdverdeling start relatief hoog, daalt daarna en stijgt vervolgens naar een stabielere waarde. De daling is te verklaren door de SRT-logica in wachtrij 0, die zeer korte processen snel verwerkt. Het budgetsysteem introduceert een kleine vertraging bij de overgang naar een lagere wachtrij, wat de hogere startwaarde en de sprong in de curve verklaart.

Round Robin

De drie RR-varianten vertonen het grootste onderlinge verschil. RR met tijdslice 2 behaalt de slechtste nTAT van alle algoritmes, RR met tijdslice 4 doet het merkelijk beter, en RR met tijdslice 8 scoort het best van de drie.

Bij een zeer korte tijdslice voert een proces tientallen contextwisselingen uit, wat de totale wachttijd en dus de nTAT sterk doet stijgen. Bij RR wacht elk proces proportioneel lang, ongeacht zijn lengte, waardoor RR systematisch hoger scoort dan algoritmes die kortere processen voorkeur geven. RR met tijdslice 8 benadert het gedrag van FCFS naarmate de tijdslice groter wordt dan de gemiddelde servicetijd, waardoor contextwisselingen zeldzamer worden en de nTAT daalt.